

ENGR1010J Introduction to Computer and Programming

Project 1 - Snake Game

Summer 2026

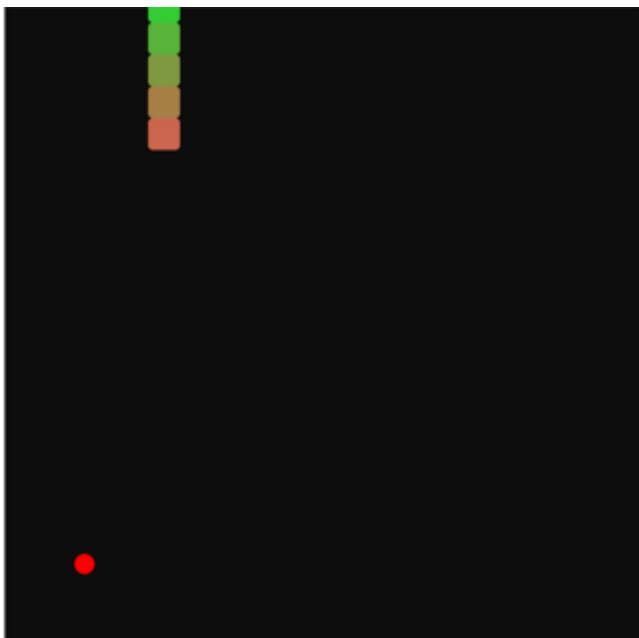
Objective

To implement a classic Snake game in MATLAB: the player controls a snake moving around on a 20×20 grid, eating apples to grow longer.

Game Rules

- The snake moves one cell per frame in the current direction.
- An apple spawns at a random empty cell. When the snake's head reaches the apple, the snake grows by one segment. A new apple then spawns at another random empty cell.
- The game ends when the snake's head:
 - Moves outside the 20×20 grid (hits a wall), or
 - Overlaps any part of its own body (hits itself).

Controls: **W** (up), **A** (left), **S** (down), **D** (right).



Save your code as **snake_game.m**.

Reference

The following code structure and development procedure serve as a reference. You may adopt your own logic and development strategy freely as long as the objective is achieved.

Figure and Keyboard Setup

Create a figure window that captures key presses via the `'KeyPressFcn'` callback:

```
fig = figure('Name', 'Snake', 'NumberTitle', 'off', ...
            'MenuBar', 'none', 'ToolBar', 'none', ...
            'KeyPressFcn', @onKeyPress);
ax = axes('Parent', fig, ...
        'XLim', [0 GRID_W], 'YLim', [0 GRID_H], ...
        'XTick', [], 'YTick', [], ...
        'Color', [0.05 0.05 0.05], ...
        'DataAspectRatio', [1 1 1]);
hold(ax, 'on');
```

The callback function receives the pressed key via `evt.Key`. Use `upper(evt.Key)` for case-insensitive matching. Store the new direction in a **buffer variable** (`next_dir`) and apply it once per frame — this prevents two rapid keypresses within one frame from causing a 180-degree reversal.

Representing the Snake

Represent the snake as an **$N \times 2$ matrix**, where each row is the `[x, y]` coordinate of one body segment (row 1 = head, row N = tail).

Use a **direction vector** `[dx, dy]` to control movement:

Key	Vector	Direction
W	<code>[0, 1]</code>	Up
S	<code>[0, -1]</code>	Down
A	<code>[-1, 0]</code>	Left
D	<code>[1, 0]</code>	Right

To prevent the snake from reversing into itself (e.g., pressing S while moving up), check that the new direction is not the opposite of the current direction before applying it.

Movement and Growth

Each frame, compute the new head as `head = snake(1, :) + dir`.

- **Normal move:** Prepend the new head and remove the tail — `snake = [head; snake(1:end-1, :)]`.
- **Eating apple:** Prepend the new head without removing the tail — `snake = [head; snake]`.

Drawing Graphics

Use `rectangle` to draw each body segment as a coloured square:

```
rectangle('Parent', ax, ...
         'Position', [x-0.5, y-0.5, 1, 1], ...
         'Curvature', [0.3 0.3], ...
         'FaceColor', 'r', ...
         'EdgeColor', 'none');
```

`Position` is `[left, bottom, width, height]`. Since each grid cell is 1x1 and centred at integer coordinates `(x, y)`, a 1x1 square centred at `(x, y)` has its bottom-left corner at `(x-0.5, y-0.5)`.

For the apple, use a plot marker:

```
plot(ax, apple_x, apple_y, 'ro', 'MarkerSize', 18, 'MarkerFaceColor', 'r');
```

To avoid flickering, update graphics each frame using `set(handle, 'Position', [...])` rather than deleting and redrawing. Use `gobjects` to preallocate graphics handle arrays.

Finding Empty Cells

Use `meshgrid` to enumerate all grid coordinates, then filter out cells occupied by the snake with `ismember(..., 'rows')`:

```
[X, Y] = meshgrid(1:GRID_W, 1:GRID_H);
all_cells = [X(:), Y(:)];
occupied = ismember(all_cells, snake, 'rows');
free = all_cells(~occupied, :);
```

Pick a random empty cell using `randi(size(free, 1))`.

Collision Detection

Two conditions end the game:

1. **Wall collision:** `head(1) < 1`, `head(1) > GRID_W`, `head(2) < 1`, or `head(2) > GRID_H`.
2. **Self collision:** `ismember(head, body(1:end-1, :), 'rows')`. The last body segment can be excluded since it moves away on the current frame.

Main Loop

Use `while running && isvalid(fig)` with `pause(SPEED)` to control the frame rate. A typical speed is around 0.12 seconds per frame.

Useful Functions

Function	Purpose
<code>meshgrid</code>	Generate all coordinate pairs on the grid
<code>ismember(..., 'rows')</code>	Check if a row exists in a matrix
<code>randi</code>	Generate random integers
<code>rectangle</code>	Draw a rectangle on axes
<code>gobjects</code>	Preallocate array of graphics handles
<code>set</code>	Modify graphics object properties
<code>pause</code>	Pause execution (controls frame rate)
<code>isvalid</code>	Check if a graphics handle is still valid
<code>upper</code>	Convert string to uppercase

Grading Policy

This project has a total of 500 points.

Source code and demonstration: 250 pts

#	Criterion	Description	Pts
1	Figure & keyboard	Figure window with dark grid; W/A/S/D keys captured and functioning; direction buffering prevents 180-degree reversal	40
2	Snake data & movement	Snake has correct head-prepend/tail-remove movement, grows by 1 after eating apple	40
3	Apple spawning	Apple spawns at random empty cell; updates after being eaten; handles full-board edge case	40
4	Collision detection	Detects wall hits (all four boundaries); detects self-collision (excluding last segment); game stops on death	40
5	Drawing & update	Snake segments drawn; apple drawn; graphics updated without flickering	30
6	Main loop	Smooth frame-by-frame movement at set speed; score display updates; game runs without errors	30
7	Demo	Runs correctly on launch; responsive W/A/S/D controls; smooth movement; score tracking works	30

Oral Examination: 250 points

Bonus (Extra Features): 50 pts

Going beyond the basic requirements can earn you **up to 50 bonus points**. Here are some ideas, but feel free to come up with your own:

- **Visual enhancements:** gradient-coloured snake body, smooth animations, particle effects when eating an apple, custom snake/apple textures.
- **Gameplay features:** increasing speed as score grows, special apples (golden apple = +3 score), obstacles/walls on the grid, wrap-around mode (snake exits one side and enters the opposite).
- **Quantity of lifes:** pause/resume with **P** key, restart after death with **R** key, high score tracking across sessions, main menu screen.
- **Sound:** sound effects for eating, death, or background music using `audioplayer/sound`.
- **Two-player mode:** a second snake controlled by **IJKL** keys on the same grid.
- **Score and Scoreboard:** a display of your score in this turn and a leaderboard of top 5 scores.

Bonus points are awarded based on creativity, technical complexity, and polish. Bonus points are awarded only when a feature is entirely implemented. So, a single well-implemented feature is worth more than several half-finished ones.

Enjoy the fun of programming and creating a game on your own!

Deliverables

Schedule a time with the TAs, show the source files with your own computer to the TAs and run them to show the output according to the TA's instructions. Be prepared for an oral exam about the project implementation. We will grade the project according to the correctness of the output, oral explanation, and the source code.

After the project, remember to compress all your files into a single zip file in the following name format: `Project1_[Your Last Name][Your First Name][Your SJTU ID].zip`. Then submit the zip file to the corresponding project link on Canvas before the due date.

Due Date and Deduction

The project should be demonstrated and the project files should be submitted no later than 11:59pm, June 22, 2026. Late submission on JOJ/Canvas will result in a deduction of 100 points per day.